Лекція №13 (2 години). Мережеві запити: Fetch API та робота з JSON.

План лекції

1. Клієнт-серверна архітектура та поняття REST API.
2. Формат обміну даними JSON.
3. Асинхронність у JS: Проблеми Callback-функцій.
4. Об'єкт Promise (Обіцянка) та його стани.
5. Сучасний синтаксис: `async` / `await`.
6. Виконання мережевих запитів за допомогою `fetch()`.
7. Обробка помилок `try...catch`.

Перелік умовних скорочень

- **API** (Application Programming Interface) — інтерфейс прикладного програмування.
- **REST** (Representational State Transfer) — архітектурний стиль для створення веб-сервісів.
- **JSON** (JavaScript Object Notation) — текстовий формат обміну даними.
- **HTTP/HTTPS** — протокол передачі гіпертексту (безпечний).

Вступ

Більшість сучасних веб-додатків не зберігають усі дані (статті, інформацію про користувачів, товари) безпосередньо в HTML-файлі. Це було б неефективно і небезпечно. Натомість, веб-сторінка (Клієнт) постійно "спілкується" з віддаленим комп'ютером (Сервером), щоб завантажити свіжі дані або відправити нові (наприклад, опублікувати коментар).

Це спілкування відбувається у фоновому режимі (без перезавантаження всієї сторінки браузера) за допомогою **мережевих запитів**. Історично для цього використовувалася технологія AJAX (XMLHttpRequest), але сьогодні стандартом став сучасний, набагато зручніший механізм — **Fetch API**.

Оскільки будь-який запит до сервера через Інтернет займає непередбачуваний час (від мілісекунд до десятків секунд у разі поганого зв'язку), програма не може просто завмерти і чекати на відповідь. JS повинен вміти обробляти ці задачі **асинхронно**. Основою асинхронності в сучасному JavaScript є `Promises` (Обіцянки) та синтаксис `async/await`.

1. Клієнт-серверна архітектура та REST API

У вебі є дві головні дійові особи:

1. **Клієнт (Front-end):** Браузер користувача з вашим HTML/CSS/JS. Він "просить" у сервера дані або просить зберегти дані.
2. **Сервер (Back-end + База даних):** Потужний комп'ютер десь у дата-центрі. Він обробляє запити, перевіряє права доступу, дістає дані з бази і відправляє їх назад клієнту.

Щоб Клієнт і Сервер розуміли один одного, вони використовують спеціальний договір-інтерфейс, який називається **API**.

Найпопулярніший формат API сьогодні — **REST API**. Він базується на використанні стандартних HTTP-методів для різних операцій (називається CRUD: Create, Read, Update, Delete):

- **GET**: Отримати дані (наприклад, список статей).
- **POST**: Створити нові дані (наприклад, зареєструвати користувача).
- **PUT / PATCH**: Оновити існуючі дані (змінити пароль).
- **DELETE**: Видалити дані з бази.

2. Формат даних JSON

Сервер може бути написаний на іншій мові програмування (Python, Java, PHP), а браузер розуміє лише JavaScript. Тому їм потрібна спільна мова для обміну даними. Сьогодні цією "мова-есперанто" є **JSON**.

Це просто довгий текстовий рядок (String), який візуально дуже нагадує звичайний об'єкт JavaScript, але з жорсткими правилами:

1. Ключі завжди беруться у **подвійні** прямі лапки `""`.
2. Не можна зберігати функції або дати як об'єкти (лише як текст).
3. Після останнього елемента не можна ставити кому.

Приклад JSON, отриманого від сервера:

```
{
  "user_id": 145,
  "name": "Ivan",
  "is_active": true,
  "hobbies": ["reading", "cycling"]
}
```

У JS є два вбудовані методи для миттєвої конвертації:

- `JSON.parse(string)` — перетворює JSON-текст у реальний JS О'бєкт.

- `JSON.stringify(object)` — перетворює JS Об'єкт у JSON-текст (для відправки на сервер).

3. Асинхронність та Проблема Callbacks

Коли ми просимо браузер завантажити дані (наприклад, велике фото), ми не знаємо, коли це закінчиться. Раніше єдиним способом відреагувати на завантаження були функції зворотного виклику (**Callbacks**):

```
// Старий, поганий код!
downloadData(
  "https://api.site.com/users",
  function (users) {
    downloadUserAvatar(
      users[0],
      function (avatar) {
        displayAvatar(
          avatar,
          function () {
            console.log("Ура, все готово");
          },
          function (error) {
            /* Обробка помилки відображення */
          },
        );
      },
      function (error) {
        /* Обробка помилки аватарки */
      },
    );
  },
  function (error) {
    /* Обробка помилки списку */
  },
);
```

Коли одна асинхронна операція залежить від результату попередньої, код перетворюється на нечитабельну "піраміду смерті" (Callback Hell).

4. Об'єкт Promise (Обіцянка)

Щоб врятувати розробників від "Піраміди смерті", в ES6 був доданий **Проміс (Promise)**. `Promise` — це спеціальний об'єкт у JS, який представляє результат асинхронної операції, який **БУДЕ** доступний у майбутньому (вона обіцяє нам повернути дані).

Проміс незримо існує в одному з трьох станів:

1. **Pending (Очікування):** Запит пішов на сервер, ми чекаємо. Це початковий стан.
2. **Fulfilled (Виконано успішно):** Сервер віддав дані без помилок (передав їх).
3. **Rejected (Відхилено з помилкою):** Сталася помилка: немає інтернету, сервер впав або код 404.

Спочатку для споживання промісів використовували методи `.then()` (якщо добре) та `.catch()` (якщо помилка).

```
// Класичне споживання Promise
fetch("https://api.test/data")
  // .then повертає новий Promise, дозволяючи будувати ланцюжки
  .then((response) => response.json())
  .then((data) => console.log("Дані отримано: ", data))
  .catch((error) => console.error("Щось пішло не так: ", error));
```

5. Сучасний синтаксис: `async / await`

Хоча `.then()` вирішив багато проблем, ланцюжки все одно були важкими для читання. З виходом ES8 розробники отримали "синтаксичний цукор" — ключові слова `async` та `await`. Це найсучасніший, найчистіший спосіб роботи з Промісами. Він дозволяє писати асинхронний код так, ніби він є звичайним синхронним (рядок за рядком).

Правила `async/await`:

1. Ви можете використовувати написати слово `await` лише всередині функції (або методу), перед якою стоїть слово `async`.
2. Слово `await` встановлюється ПЕРЕД промісом. Воно змушує JS-код всередині цієї функції "стати на паузу" і дочекатися, поки Promise перейде у стан `Fulfilled`. Після цього воно поверне реальні дані (без всяких `then`).

```
/* Сучасний, красивий код: */

async function loadUserData() {
```

```
  console.log("Починаємо...");

  // JS робить паузу (всередині функції) і чекає на відповідь сервера
  const response = await fetch("https://jsonplaceholder.typicode.com/users/1");

  // JS розуміє: ага, response вже тут
  console.log("Відповідь є!");

  // Нам треба почекати парсинг JSON (він теж асинхронний!)
  const user = await response.json();

  console.log(`Привіт, ${user.name}!`);
}

loadUserData();
```

6. Виконання запитів: `fetch()`

Функція `fetch(url, [options])` вбудована у всі сучасні браузери. Її призначення — робити HTTP-запити. Вона "під капотом" завжди повертає Promise.

Якщо ви викликаєте `fetch()` з одним аргументом (тільки URL) — він завжди створює стандартний **GET-запит** (тобто просить віддати йому інформацію).

Особливість: `fetch` не вважає помилки з кодом 404 (Не знайдено) або 500 (Ой, сервер зламався) серйозними (Rejected). Для `fetch` відхилений проміс — це лише ситуація, коли взагалі немає інтернету і він не зміг "достукатися" до сервера. Тому розробники завжди повинні вручну перевіряти властивість `response.ok`. Повертає `true` (якщо код 200..299).

Відправка даних на сервер (Метод POST)

Щоб не тільки прочитати, але й зберегти нові дані (наприклад, форму реєстрації), ми маємо передати у `fetch` другий аргумент — об'єкт з налаштуваннями (`options`):

```
async function createPost(postTitle, postBody) {
  const newPost = {
    title: postTitle,
    body: postBody,
    userId: 1,
  };
```

```
  // Об'єкт налаштувань
  const options = {
    method: "POST", // Вказуємо, що ми створюємо, а не читаємо
    headers: {
      // Критично важливо сказати серверу, який саме формат ми зараз надішлемо
      "Content-Type": "application/json",
    },
    // Перетворюємо реальний об'єкт JS у текстовий JSON-код!
    body: JSON.stringify(newPost),
  };

  // Відправляємо
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts",
    options,
  );

  if (response.ok) {
    const savedData = await response.json();
    console.log("Успішно створено з ID:", savedData.id);
  }
}
```

7. Обробка помилок `try...catch`

Оскільки при використанні `async/await` ми відмовилися від блоку `.catch()`, виникає питання: "А що буде, якщо інтернет зникне на середині завантаження?". Сценарій видасть неконтрольовану помилку (Uncaught Promise Rejection) і додаток може зламатися.

Щоб цьому запобігти, ВЕСЬ асинхронний код всередині `async`-функції потрібно огортати в класичний блок `try...catch`.

```
async function getWeather(city) {
  try {
    // Пробуємо виконати небезпечний блок...
    const response = await fetch(`https://weather-api.com/${city}`);

    // Вручну кидаємо помилку, якщо щось не так на стороні їхнього сервера
    if (!response.ok) {
```

```
      throw new Error(`Хмарно з проясненнями багів: статус ${response.status}`);
    }

    const data = await response.json();
    return data.temperature;
  } catch (error) {
    // Якщо ВСЮДИ вище сталася технічна або мануальна помилка — код перескакує
сюди!
    console.error("Сталася катастрофа:", error.message);

    // Можна показати гарне повідомлення-помилку користувачу (Тостер / Модалка)
    // ui.showError(error.message);
  }
}
```

Висновки

1. Сучасний веб працює на взаємодії браузера (Клієнта) з сервером через REST API за допомогою мережевих HTTP-запитів по протоколу HTTPS.
2. JSON — це єдиний універсальний текстовий формат "розмови" між JS та будь-якою іншою мовою бек-енду на сервері. Для конвертації туди-сюди використовують `JSON.stringify` та `JSON.parse` (або `response.json()`).
3. Виконання запиту в Інтернет — процес асинхронний. Результат цього процесу в JS репрезентується об'єктом **Promise** (Обіцянка).
4. Сучасний і чистий спосіб дочекатися виконання обіцянки, не плодячи Callback-функції та довгі ланцюги `.then()` , — це вказати перед викликом слово `await` (вкладеним у функцію з тегом `async`).
5. Вбудована функція `fetch` — головний інструмент веб-розробника. Без налаштувань вона використовує метод `GET` , але через додатковий об'єкт `options` її перетворюють на агрегатор для `POST` , `PUT` , `DELETE` .
6. Асинхронний код завжди потенційно ризикований (інтернет-з'єднання може обірватись). Комплекс `try...catch` гарантує обробку помилок і захищає додаток від повного "падіння".

Джерела

1. MDN Web Docs: Using Fetch
2. Javascript.info: Fetch - детально українською.
3. Javascript.info: Async/await

4. JSON.org - офіційно про стандарт JSON.

Запитання для самоперевірки

1. Чим відрізняється HTTP Метод `GET` від методу `POST` у концепції REST API? Який з них викликається функцією `fetch()` за замовчуванням?
2. Вкажіть три правила (відмінності), які відрізняють формат тексту JSON від класичного об'єкта JavaScript у пам'яті?
3. Що таке Promise в JavaScript, і в яких трьох станах він може перебувати?
4. Яка роль ключового слова `await`, чому без нього функція `loadData()` видала б помилку, або просто повернула Promise `Pending`? Де саме його потрібно ставити?
5. Напишіть рядок коду (лише рядок), який перетворить отриманий по HTTP текстовий JSON із масивом у реальний масив даних JS.
6. Навіщо у конфігурації (в `options`) при здійсненні `POST`-запиту ми зобов'язані передати параметр headers `Content-Type: application/json`?
7. Чому функція `fetch` може працювати без генерації системної помилки ігноруючи навіть статус "404 Not Found"? Як з цим боротися?
8. Яке призначення блоку `try...catch` при написанні функції з мережевими запитами? Що конкретно виконує секція `catch(error)`?